Übungsblatt 5 mit Lösungen

Aufgabe 5.1 (Chat-Server mit TCP)

Erweitern Sie Ihr Programm aus Aufgabe 4.4 derart, dass Ihr Server mehrere Verbindungen gleichzeitig offen halten kann. Für jede offene Verbindung soll der Server sich den Benutzernamen merken. Bei Eingang einer neuen Verbindung sollen an die entsprechende Gegenstelle alle bisher bekannten Benutzernamen und an alle bereits offenen Verbindungen ein Hinweis auf den neuen Teilnehmer verschickt werden. Entsprechend sollen beim Abbau einer Verbindung die noch offenen Teilnehmer über den Weggang des Teilnehmers informiert werden. Des weiteren sollen alle Daten, die auf einer Verbindung eingehen, an alle offenen Verbindungen (inklusive der Quelle) verschickt werden.

Entwickeln Sie dazu ein (wie auch immer geartetes) Protokoll basierend auf TCP, mit dem Sie im Client programmatisch (i.e. ohne „Benutzerintelligenz“) eine Liste der aktuell aktiven Gesprächsteilnehmer führen können.

Fordern Sie serverseitig einen nicht-leeren Benutzernamen und verweigern Sie die gleichzeitige mehrfache Verwendung eines Benutzernamens.

```
package smf.ves;
import java.io.*;
import java.net.*;
import java.util.Vector;
public class Aufg5_1_ChatServer extends Thread {
  public final static int PORT = 4711;
  public static void main(String[] args) {
    ServerSocket serverSocket=null;
    Vector<Socket> clients=new Vector<Socket>();
    Vector<BufferedWriter> clientOuts=new Vector<BufferedWriter>();
    Vector<String> users=new Vector<String>();
    try {
      //create the server, implicitly bind to given port
      serverSocket=new ServerSocket(PORT);
      //set timeout for blocking accept
      serverSocket.setSoTimeout(1000);
      System.out.println("--- Started threaded server on port "
        +serverSocket.getLocalPort()+", waiting (end with new line)");
      //wait for client to connect
      int cnt=0;
      while (System.in.available()==0) {
        try {
```

```
        Socket client=serverSocket.accept();
        System.out.println("--- Got connection from "+client.getInetAddress()
          +" at port "+client.getPort()+", starting new Thread");
        (new Aufg5_1_ChatServer(cnt++, client,
            clients, clientOuts, users)).start();
      }
      catch (SocketTimeoutException e) { /* not a problem, check again */ }
    }
    while (System.in.available()>0) System.in.read(); //clean up
  }
  catch (IOException e) {
    System.err.println("there was an exception in main: "+e.getMessage());
  }
  if (serverSocket!=null) {
    try {
      //close the server socket
      serverSocket.close();
      System.out.println("--- Server stopped");
    }
    catch (IOException e) { /* can't do anything now */ }
  }
}

//thread to receive data from a single client
//  and to process this data which may cause sending data to other clients
//  therefore everything has to be synchronized through an object
//the client must send "usr:" and his username in the first line
//  every following line will contain text entered by this particular user
//we will send received text messages to all clients with a "txt:" header
//  and tell clients, if a new client joins ("add:") or leaves ("del:")
//if a username is already given, the client is kicked ("dny!")

private int threadNumber;
private Socket localClient;
private BufferedWriter out;
private Vector<Socket> globalClients;
private Vector<BufferedWriter> globalClientOuts;
private Vector<String> globalUsers;
```

```
  private Aufg5_1_ChatServer(int number, Socket client,
      Vector<Socket> clients, Vector<BufferedWriter> clientOuts,
      Vector<String> users) {
    threadNumber=number;
    localClient=client;
    globalClients=clients;
    globalClientOuts=clientOuts;
    globalUsers=users;
  }
  public void run() {
    String localUser=null;
    try {
      String line;
      //get handles to input and output streams
      BufferedReader in=new BufferedReader(
        new InputStreamReader(localClient.getInputStream()));
      out=new BufferedWriter(
        new OutputStreamWriter(localClient.getOutputStream()));
      //get username from in-stream
      if ((line=in.readLine())==null || !line.startsWith("usr:")
          || (localUser=line.substring(4))==null || localUser.length()<1) {
        System.err.println("invalid user information in connection from "
          +localClient.getInetAddress());
        return;
      }
      synchronized (globalClients) {
        //check if user is already existing
        for (String u: globalUsers) if (localUser.equals(u)) {
          out.write("dny!");
          out.newLine();
          out.flush();
          System.out.println("--- "+threadNumber
            +" Force client shutdown (user already existing)");
          localClient.close();
          return;
        }
        //thread data ok
        System.out.println("--- Started thread "+threadNumber);
        //send the list of current members to the new client
```

```
    for (String u: globalUsers) {
      out.write("add:");
      out.write(u);
      out.newLine();
    }
    out.flush();
    //inform other clients and add local client with output stream
    for (BufferedWriter o: globalClientOuts) {
      o.write("add:");
      o.write(localUser);
      o.newLine();
      o.flush();
    }
    globalClients.add(localClient);
    globalClientOuts.add(out);
    globalUsers.add(localUser);
  }
  //loop to get data of this client and send every received line
  while ((line=in.readLine())!=null) {
    System.out.println("--- "+threadNumber+" Received: "+line);
    synchronized (globalClients) {
      for (BufferedWriter o: globalClientOuts) {
        o.write("txt:");
        o.write(localUser);
        o.write(':');
        o.write(line);
        o.newLine();
        o.flush();
      }
    }
  }
  //client has closed his side, inform other clients
  //and remove and close the client connection socket
  synchronized (globalClients) {
    globalClients.remove(localClient);
    globalClientOuts.remove(out);
    globalUsers.remove(localUser);
    for (BufferedWriter o: globalClientOuts) {
      o.write("del:");
```

```
          o.write(localUser);
          o.newLine();
          o.flush();
        }
      }
      System.out.println("--- "+threadNumber+" Client shutdown");
      localClient.close();
    }
    catch (IOException e) { //handle the unlikely case of an IO-exception
      System.err.println("there was an exception at the server thread "
        +threadNumber+": "+e.getMessage());
      globalClients.remove(localClient);
      globalClientOuts.remove(out);
      globalUsers.remove(localUser);
    }
  }
}
```

Aufgabe 5.2 (Chat-Client mit TCP und GUI)

Erweitern Sie Ihr Programm aus Aufgabe 4.6 derart, dass Ihr Programm das von Ihnen in Aufgabe 5.1 entwickelte Protokoll versteht, im Chat-Fenster den Beitritt und Austritt von Teilnehmern vermerkt und eine Liste der aktuellen Gesprächsteilnehmer anzeigt.

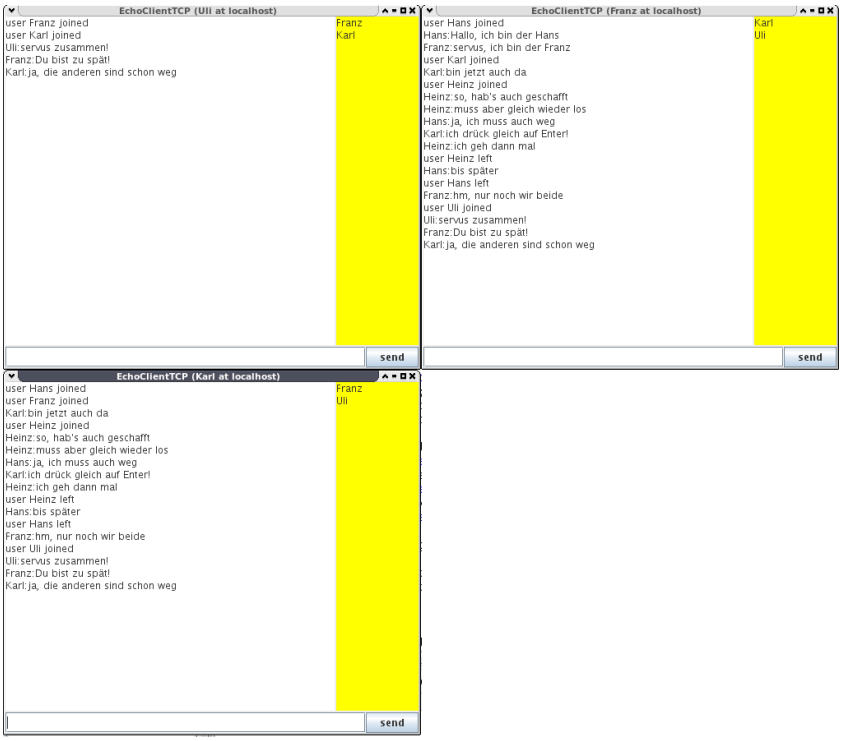
```
package smf.ves;
import java.awt.*;
import java.awt.event.*;
import java.io.*;
import java.net.*;
import java.util.Vector;
import javax.swing.*;

public class Aufg5_2_ChatClient extends JFrame implements ActionListener {
  private static final long serialVersionUID = 1L;
  public final static int SERVER_PORT = 4711;
  public final static String PROG_NAME = "EchoClientTCP";
  private JTextArea text;
  private JTextArea usersText;
  private JTextField inputField;
  private JButton sendButton;
  private Socket socket;
```

```
  private BufferedWriter outStream;
  private Vector<String> users;
 public static void main(String[] args) {
    final String server, user;
    //get server- and user-name
    if ((server=(String)JOptionPane.showInputDialog(null,
        "Please enter your chat server", PROG_NAME,
        JOptionPane.QUESTION_MESSAGE, null, null, "localhost"))==null
        || "".equals(server)) return;
    if ((user=(String)JOptionPane.showInputDialog(null,
        "Please enter your user name", PROG_NAME,
        JOptionPane.QUESTION_MESSAGE, null, null, "smf"))==null
        || "".equals(user)) return;
    //create the socket, initialize connection
    final Socket preparedSocket;
    final BufferedWriter preparedOutStream;
    try {
      //server requires us to send a line containing "usr:" and the username
      preparedSocket=new Socket();
            preparedSocket.connect(new  InetSocketAddress(server,  SERVER_PORT),
1000); //set 1s timeout
      preparedOutStream=new BufferedWriter(
        new OutputStreamWriter(preparedSocket.getOutputStream()));
      preparedOutStream.write("usr:");
      preparedOutStream.write(user);
      preparedOutStream.newLine();
      preparedOutStream.flush();
    }
    catch (SocketException e) {
      JOptionPane.showMessageDialog(null, "Error creating socket");
      return;
    }
    catch (UnknownHostException e) {
      JOptionPane.showMessageDialog(null, "Could not resolve host "+server);
      return;
    }
    catch (IOException e) {
      JOptionPane.showMessageDialog(null, "I/O-error while connecting to host");
      return;
```

```
    }
    //start the GUI
    SwingUtilities.invokeLater(new Runnable() {
      public void run() {
        new Aufg5_2_ChatClient(server, user, preparedSocket, preparedOutStream);
      }
    });
  }

  public Aufg5_2_ChatClient(String server, String user,
      Socket preparedSocket, BufferedWriter preparedOutStream) {
    //display a nice title bar
    super(PROG_NAME+" ("+user+" at "+server+")");
    //remember connection specific things
    socket=preparedSocket;
    outStream=preparedOutStream;
    users=new Vector<String>();
    //add components to GUI, set default behavior
    setDefaultCloseOperation(EXIT_ON_CLOSE);
    setLayout(new BorderLayout(2, 2));
    add(text=new JTextArea(), BorderLayout.CENTER);
    text.setPreferredSize(new Dimension(400, 400));
    add(usersText=new JTextArea(), BorderLayout.EAST);
    usersText.setPreferredSize(new Dimension(100, 400));
    usersText.setEditable(false);
    usersText.setBackground(Color.YELLOW);
    JPanel bottom=new JPanel();
    bottom.setLayout(new BorderLayout());
    bottom.add(inputField=new JTextField(), BorderLayout.CENTER);
    inputField.addActionListener(this);
    bottom.add(sendButton=new JButton("send"), BorderLayout.EAST);
    sendButton.addActionListener(this);
    add(bottom, BorderLayout.SOUTH);
    //pack the components and display the window , set focus to input field
    pack();
    setVisible(true);
    inputField.requestFocus();
    //create and start a client updater thread
    (new ChatClient_Updater()).start();
```

```
  }

  //handler for actions from inputField and sendButton
  //  just send the entered text to the server
  //  all other information is organized there
  public void actionPerformed(ActionEvent event) {
    String line;
    if ((line=inputField.getText()).equals("")) { //empty inputField
      JOptionPane.showMessageDialog(this, "Stopping program after empty line");
      System.exit(0);
    }
    try { //send the entered text, send a new line, and flush the TCP-buffer
      outStream.write(line);
      outStream.newLine();
      outStream.flush();
    }
    catch (IOException e) {
      JOptionPane.showMessageDialog(null, "I/O-error while sending data");
      System.exit(-1);
    }
    //clear the inputText as the user will start typing without clearing
    inputField.setText("");
  }

  //thread to receive packets
  //  all data is transfered in lines
  //  each line has a four-character-prefix:
  //    txt: append following characters as text
  //    add: add new user
  //    del: remove user from list
  //    dny! server denies participation
  private class ChatClient_Updater extends Thread {
    public void run() {
      BufferedReader in;
      try {
        in=new BufferedReader(new InputStreamReader(socket.getInputStream()));
        while (true) {
          String line;
          if ((line=in.readLine())==null) {
```

```
          throw new IOException("Server closed connection");
        }
        if (line.length()<4) throw new IOException("Invalid server data");
        String cmd=line.substring(0, 4), data=line.substring(4);
        //JTextArea.append thread safe, no need of SwingUtilities.invokeLater
        if (cmd.equals("txt:")) { //print text
          text.append(data);
          text.append("\n");
        }
        else if (cmd.equals("add:")) { //add entry in user-list
          text.append("user ");
          text.append(data);
          text.append(" joined\n");
          users.add(data);
          usersText.append(data);
          usersText.append("\n");
        }
        else if (cmd.equals("del:")) { //remove entry in user-list
          text.append("user ");
          text.append(data);
          text.append(" left\n");
          usersText.setText("");
          for (int i=users.size()-1; i>=0; i--) {
            String u;
            if ((u=users.get(i)).equals(data)) users.remove(i);
            else {
              usersText.append(u);
              usersText.append("\n");
            }
          }
        }
        else if (cmd.equals("dny!")) { //we are kicked
          JOptionPane.showMessageDialog(null, "Server denied connection");
          System.exit(0);
        }
        else throw new IOException("Unknown server command");
      }
    }
    catch (IOException e) {
```

```
            JOptionPane.showMessageDialog(null, "I/O-error while receiving data: "
                +e.getMessage());
            System.exit(-1);
        }
    }
  }
}
```

Beispiel einer Sitzung mehrerer Benutzer an einem Rechner